

Desenvolvimento de Software
Davi Santos
Gabriel Schons
Gabriel Copatti
Julio Augusto de Castilhos Borges

INF01120
00599530
00342020
00342808
00590537

Lista de Requisitos Funcionais

O sistema deve:

1. Fornecer um campo de entrada de texto para o usuário inserir ou colar sequências de caracteres.
2. Mapear caracteres específicos para instruções musicais definidas no sistema.
3. Permitir que o usuário defina parâmetros iniciais de reprodução através da interface, incluindo BPM, instrumento inicial, oitava padrão e volume padrão.
4. Sintetizar e reproduzir a sequência de áudio resultante no dispositivo de áudio padrão do sistema.
5. Exibir uma interface gráfica com controles para ajuste de parâmetros em tempo real.
6. Permitir que o usuário inicie, pause e reinicie a reprodução da sequência musical.
7. Validar a entrada de texto e fornecer feedback ao usuário sobre caracteres não reconhecidos.
8. Manter o estado atual do sistema, incluindo BPM, volume, oitava e instrumento ativo.

Requisitos Não Funcionais

1. O sistema deve seguir os princípios SOLID de orientação a objetos.
2. O código fonte deve ser gerenciado através do sistema de controle de versão Git.
3. O sistema deve ser responsivo e não bloquear a interface durante a síntese de áudio.

Definição das Classes

Classe MusicMachine

Atributos:

- currentBPM: int - Armazena o tempo atual em batidas por minuto
- currentVolume: float - Armazena o volume atual (0.0 a 1.0)
- currentOctave: int - Armazena a oitava atual
- currentInstrument: string - Armazena o instrumento ativo
- audioEngine: Audio - Referência para o motor de áudio
- tokenProcessor: TokenProcessor - Referência para o processador de tokens

Métodos:

- play(sequence: string): void - Inicia a reprodução da sequência musical
- pause(): void - Pausa a reprodução atual
- reset(): void - Reinicia todos os parâmetros para os valores padrão
- setBPM(bpm: int): void - Define o tempo em batidas por minuto
- setVolume(volume: float): void - Define o volume de reprodução
- setOctave(octave: int): void - Define a oitava padrão
- setInstrument(instrument: string): void - Define o instrumento ativo

Classe Audio

Atributos:

- `sampleRate: int` - Taxa de amostragem do áudio
- `bufferSize: int` - Tamanho do buffer de áudio
- `isPlaying: bool` - Indica se o áudio está sendo reproduzido

Métodos:

- `initialize(): bool` - Inicializa o motor de áudio
- `playNote(frequency: float, duration: float, instrument: string): void` - Reproduz uma nota musical
- `stop(): void` - Interrompe a reprodução atual
- `setSampleRate(rate: int): void` - Define a taxa de amostragem
- `shutdown(): void` - Finaliza o motor de áudio

Classe GUI

Atributos:

- `musicMachine: MusicMachine` - Referência para a máquina musical
- `textInput: TextField` - Campo de entrada de texto
- `bpmSlider: Slider` - Controle deslizante para BPM
- `volumeSlider: Slider` - Controle deslizante para volume
- `octaveSelector: ComboBox` - Seletor de oitava
- `instrumentSelector: ComboBox` - Seletor de instrumento
- `playButton: Button` - Botão de reprodução
- `pauseButton: Button` - Botão de pausa
- `resetButton: Button` - Botão de reinício

Métodos:

- `initialize(): void` - Inicializa a interface gráfica
- `onTextChanged(text: string): void` - Manipula mudanças no texto de entrada
- `onBPMChanged(value: int): void` - Manipula mudanças no BPM
- `onVolumeChanged(value: float): void` - Manipula mudanças no volume
- `onPlayClicked(): void` - Manipula o clique no botão de reprodução
- `onPauseClicked(): void` - Manipula o clique no botão de pausa
- `onResetClicked(): void` - Manipula o clique no botão de reinício
- `updateDisplay(): void` - Atualiza a exibição da interface

Classe TokenProcessor

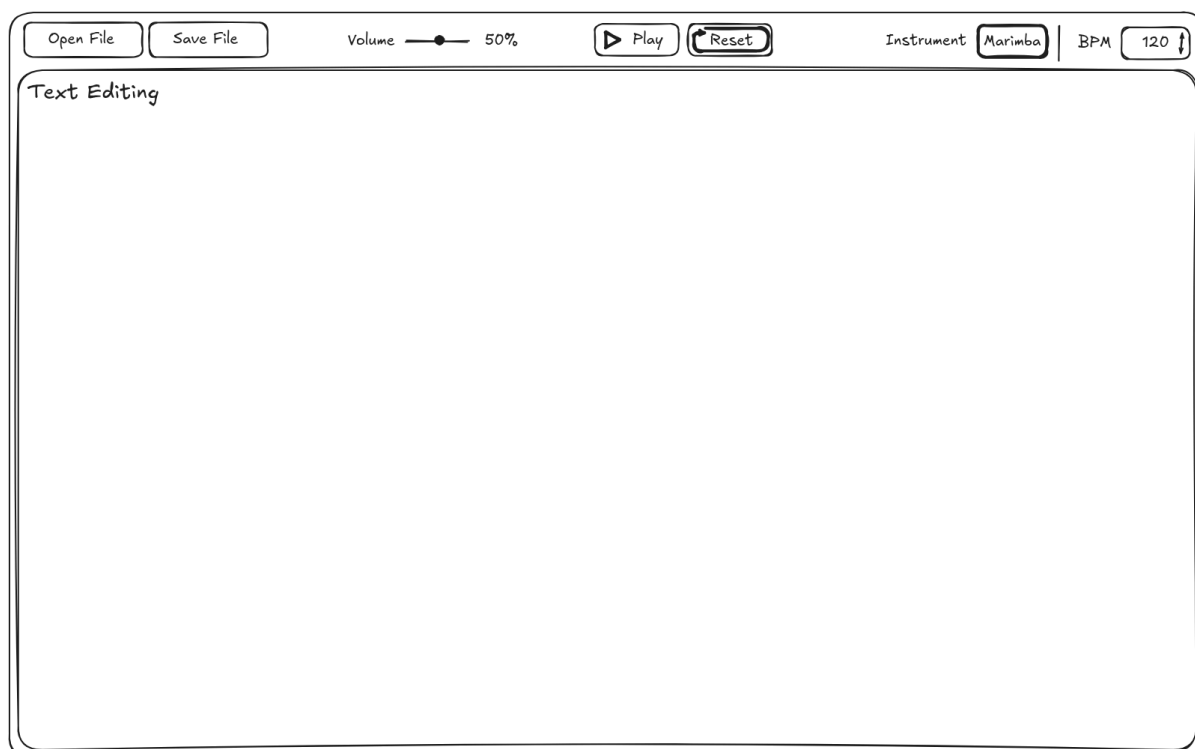
Atributos:

- `mappingRules: dict` - Dicionário de regras de mapeamento caractere-instrução
- `currentPosition: int` - Posição atual na sequência de texto

Métodos:

- `tokenize(input: string): list` - Realiza a análise léxica da string de entrada
- `addMappingRule(character: char, instruction: Instruction): void` - Adiciona uma nova regra de mapeamento
- `removeMappingRule(character: char): void` - Remove uma regra de mapeamento existente
- `validateInput(input: string): bool` - Valida a entrada do usuário
- `getInvalidCharacters(input: string): list` - Retorna caracteres não reconhecidos na entrada

Croqui da Interface com Usuário (GUI)



Arquitetura do Sistema

O sistema deve ser estruturado em torno de um design modular e orientado a objetos que separe estritamente as responsabilidades entre a camada de apresentação (GUI) e a lógica de negócio principal (camada tipo API). Para garantir manutenibilidade, extensibilidade e aderência aos princípios SOLID—especificamente Inversão de Dependência e Aberto/Fechado—as seguintes classes compõem a aplicação:

- **MusicMachine:** Atua como o orquestrador central do sistema, responsável por coordenar interações entre todos os subsistemas, gerenciar o estado compartilhado (por exemplo, BPM atual, volume, oitava, instrumento ativo) e delegar fluxos de execução para componentes especializados. Esta classe serve como a interface primária para a GUI acionar a reprodução, reiniciar parâmetros ou consultar o status do sistema.
- **Audio:** Encapsula todas as interações com o backend de síntese de áudio, fornecendo uma interface abstrata para geração de som, gerenciamento de instrumentos e ajustes de parâmetros em tempo real. Ao depender de abstrações em vez de implementações concretas, esta classe permite a substituição futura do motor de áudio sem impactar módulos de nível superior.
- **GUI:** Implementa a interface do usuário, manipulando eventos de entrada do usuário (operações de arquivo, ajustes de parâmetros, controles de reprodução), atualizando elementos visuais em resposta a mudanças de estado do sistema e encaminhando ações do usuário para o MusicMachine através de sinais e slots bem definidos. Esta classe permanece desacoplada da lógica de negócio, dependendo apenas de interfaces expostas pela camada principal.
- **TokenProcessor** (título provisório para “Módulo 1”): Responsável pela análise léxica da string de entrada, mapeando caracteres para comandos semânticos de acordo com regras configuráveis e emitindo eventos estruturados para o MusicMachine executar. Projetado de acordo com o Princípio Aberto/Fechado, esta classe permite que novas regras de mapeamento texto-música

sejam adicionadas através de extensão em vez de modificação do motor de processamento principal.

Todas as classes concretas devem depender de interfaces abstratas, permitindo injeção de dependência e facilitando testes unitários. A arquitetura garante que novos recursos—como mapeamentos adicionais de tokens, backends de áudio alternativos ou componentes de UI estendidos—possam ser integrados com impacto mínimo no código existente, promovendo manutenibilidade e escalabilidade a longo prazo.